

ASTU Bank Desktop Application

How the code works (step-by-step technical walkthrough)

This document explains the flow of the Java program from startup through login, banking actions, and saving data. Class names match the packages under `src/main/java/banking/`.

1. Big picture (layers)

The program is split into five ideas that work together.

- Model (`banking.model`): plain Java objects that hold account data and transactions. They do not talk to the screen or files by themselves.
- Store (`banking.store`): reads and writes the whole account database as one serialized file (`data/bank_data.ser`).
- Service (`banking.service`): the rules engine. The UI calls `BankService`; the service updates `UserAccount` objects and tells the store to save.
- Security (`banking.security`): turns passwords into SHA-256 hashes so the program never stores raw passwords.
- UI (`banking.ui`): Swing windows. It collects clicks and text, then calls `BankService` and shows messages.

Flow in one sentence: Main starts the UI; the UI calls `BankService`; `BankService` uses in-memory accounts loaded from `BankDataStore`; after changes, `BankService` saves back to disk and optional text files.

2. Step-by-step: program starts (`Main.java`)

Step 1: `main()` runs.

Step 2: `new BankDataStore("data/bank_data.ser")` remembers where the database file lives.

Step 3: `new BankService(dataStore)` loads all accounts from that file into a Map in memory. If the file is missing, the map starts empty.

Step 4: If there are no accounts at all, `BankService` creates three demo users (`UGR/00001/24`, etc.) and saves them.

Step 5: `new BankingAppUI(bankService)` keeps a reference to the service so every button can use it.

Step 6: `show()` schedules the login window on the Swing event thread (`invokeLater`).

3. Step-by-step: loading and saving data (`BankDataStore.java`)

`loadAccounts()`:

- If `data/bank_data.ser` does not exist, return an empty `LinkedHashMap`.
 - Otherwise open an `ObjectInputStream`, read one object, and if it is a Map of String to `UserAccount`, use it.
- On any error, return empty (safe fallback).

`saveAccounts(accounts)`:

- Create parent folders if needed.
- Write the entire Map with `ObjectOutputStream` (Java serialization). `UserAccount` and `Transaction` must be `Serializable` (they are).

This means all accounts live in one file; load once at startup, save after each business operation that changes data.

4. Step-by-step: BankService as the brain

The service holds accounts in a Map keyed by student ID (normalized to upper case). Public methods are synchronized so two Swing actions cannot corrupt the map at the same time.

normalizeStudentId: trims and uppercases the ID.

isValidStudentId: checks the pattern UGR/five digits/two digits.

authenticate(studentId, password):

- Find the account. If missing, return null.
- Hash the typed password and compare to stored hash. Return the account only if they match.

register: validates ID, names, password length, checks ID not already used, creates UserAccount with hashed password, adds an opening deposit transaction, put in map, persist.

deposit / withdraw: find account, check amount > 0, update balance, append a Transaction, persist. Withdraw also checks balance before subtracting.

transfer: validate receiver ID, load sender and receiver, prevent self-transfer, check balance, subtract from sender and add to receiver, two transaction records, persist.

changePassword: verify old password hash, set new hash, persist.

getBalance / getTransactions / getFirstName: read-only helpers for the UI.

persist(): calls datastore.saveAccounts and then savePerUserFiles() which writes one simple .txt per user under data/users/ for easy inspection (not the primary database).

5. Step-by-step: model classes

UserAccount: username, first/last name, password hash, balance, list of Transaction. deposit() and withdraw() only change the number; the service decides rules. addTransaction() records history.

Transaction: records type (enum TransactionType), amount, note, and balance after the operation.

TransactionType: values such as DEPOSIT, WITHDRAWAL, TRANSFER_IN, TRANSFER_OUT.

6. Step-by-step: passwords (PasswordUtil.java)

hash(password): SHA-256 over UTF-8 bytes, turned into a hex string.

matches(plainPassword, storedHash): hash the plain text and compare strings. Login and password confirmation both use this pattern.

7. Step-by-step: login and registration UI (BankingAppUI)

buildLoginFrame builds the blue background panel and the rounded white card.

When the user presses Log in:

1. Read Student ID field; BankService.normalizeStudentId fills formatting.
2. If ID format invalid, show error dialog and stop.
3. Read password; call authenticate. If null, invalid ID or password.
4. On success, remember currentUsername and first name, dispose login frame, open dashboard buildDashboardFrame().

Registration opens a modal dialog; on Create it validates fields and calls `bankService.register(...)`.

Logout from the dashboard disposes the dashboard and rebuilds the login frame.

8. Step-by-step: dashboard actions

Deposit card: parses amount; calls `deposit`; `refreshBalance` and `refreshTransactions` update labels and table.

Withdraw card: parses amount; shows a confirm dialog with password; `authenticate` must succeed; then `withdraw` + `refresh`.

Transfer tab: normalizes recipient ID, parses amount, `bankService.transfer`, then `refresh`.

Security tab: `changePassword` with old and new passwords.

Transactions tab: table filled from `bankService.getTransactions`.

9. End-to-end example: successful withdrawal

- 1 User enters amount and clicks Withdraw.
- 2 UI parses amount; if invalid, message and exit.
- 3 UI shows password dialog; user cancels then nothing happens.
- 4 UI calls `authenticate(currentUser, password)`; if wrong, error message and exit.
- 5 UI calls `bankService.withdraw(currentUser, amount)`.
- 6 Service finds account, checks `amount > 0` and `amount <= balance`.
- 7 Service calls `account.withdraw(amount)` and `addTransaction(WITHDRAWAL, ...)`.
- 8 Service calls `persist()`: saves `bank_data.ser` and rewrites `data/users/*.txt` files.
- 9 UI shows the returned string (for example `Withdrawal successful.`) and refreshes screen data.

10. Files to show a teacher

`Main.java`: entry point.

`BankService.java`: all business rules in one place.

`BankDataStore.java`: serialization load/save.

`UserAccount.java`, `Transaction.java`: data structure.

`PasswordUtil.java`: hashing.

`BankingAppUI.java`: Swing screens and how they call the service.

Runtime data: `data/bank_data.ser` (binary) and `data/users/` (human-readable exports).